

BUENAS PRÁCTICAS DE PROGRAMACIÓN EN SISTEMAS EMBEBIDOS

GUÍA METODOLÓGICA DE OPTIMIZACIÓN DE MEMORIA RAM, FLASH
Y COMPILACIÓN EFICIENTE EN VS CODE Y CMAKE

Autor:
Rodrigo CASTILLEJOS

Proyecto:
Control de Dispositivos Embebidos

June 9, 2026

RESUMEN

Este documento técnico recopila las mejores prácticas de programación en C aplicadas al desarrollo de sistemas embebidos con recursos limitados. Debido a la limitada capacidad de memoria RAM y almacenamiento Flash física en los microcontroladores (como las series STM32), el firmware debe diseñarse con técnicas rigurosas de optimización. Se analizan temas críticos tales como la mitigación de fragmentación eliminando la asignación dinámica, el uso correcto de tipos de datos de ancho fijo (`<stdint.h>`), el almacenamiento de constantes en Flash mediante el calificador *const*, el control del Stack, el paso por referencia y el uso del calificador *volatile* ante registros e interrupciones. Por último, se proporciona una guía detallada sobre cómo configurar las opciones de optimización del compilador GCC (especialmente el flag `-Os`) en entornos de desarrollo basados en VS Code, utilizando CMake, Makefiles y archivos de configuración de tareas.

TABLA DE CONTENIDOS

1	Introducción a las Restricciones en Sistemas Embebidos	4
2	Arquitectura de Memoria y el Bootloader	4
2.1	Clasificación de Memorias Físicas y sus Funciones	4
2.1.1	Memorias Auxiliares del Sistema	5
2.2	El Bootloader y su Relación con la Memoria Flash	5
3	Buenas Prácticas para la Gestión de Memoria RAM	6
3.1	Prohibición Absoluta de la Asignación Dinámica	6
3.1.1	Consecuencias de la asignación dinámica	6
3.2	Optimización de Tipos de Datos y Ancho de Palabra (<stdint.h>)	7
3.3	Estructuras y Alineación de Memoria	8
3.4	Prevención de Desbordamiento del Stack y Análisis de Uso	9
3.4.1	Habilitación de -fstack-usage en CMake y VS Code	10
3.5	Paso de Parámetros por Referencia (Punteros) y Uso de Const	10
4	Optimización de Almacenamiento Flash (ROM) y Rendimiento	11
4.1	Almacenamiento de Constantes en Flash mediante const	11
4.2	Evitar la Aritmética de Punto Flotante y Funciones Pesadas	11
4.2.1	Reglas para optimizar cálculos aritméticos:	11
5	Uso del Calificador volatile	14
5.1	¿Cuándo debe utilizarse?	14
5.2	Análisis del código Ensamblador generado	14
6	Optimización del Compilador GCC en VS Code	16
6.1	Niveles de Optimización en GCC	16
6.2	La Bandera de Optimización de Tamaño: -Os	16
6.3	Cómo configurar la optimización en entornos de VS Code	17
6.3.1	Caso A: Proyectos basados en CMake (STM32 VS Code Extension)	17
6.3.2	Caso B: Proyectos basados en Makefile tradicionales	17
6.3.3	Caso C: Tareas integradas de compilación en VS Code (tasks.json)	18
6.4	Banderas adicionales de optimización para reducir tamaño	18
6.4.1	Configuración de eliminación de código muerto en CMake	19
6.5	Análisis del uso de memoria en VS Code	19

1 INTRODUCCIÓN A LAS RESTRICCIONES EN SISTEMAS EMBEBIDOS

A diferencia de los sistemas de cómputo de propósito general, los microcontroladores embebidos operan bajo severas restricciones físicas de hardware. Mientras que una computadora común dispone de gigabytes de memoria de acceso aleatorio (RAM) y almacenamiento en estado sólido, un microcontrolador de rango medio típicamente posee entre unos cuantos kilobytes hasta un par de megabytes de memoria Flash y de escasos kilobytes de RAM física.

Esta arquitectura física determina que el desarrollo de firmware requiera un diseño minucioso y óptimo. Un byte desperdiciado en una estructura mal alineada o la importación innecesaria de una librería matemática pesada puede significar que el ejecutable supere los límites físicos del chip, impidiendo su compilación o provocando fallos fatales en tiempo de ejecución (como bloqueos de tipo *HardFault* o desbordamientos del Stack).

2 ARQUITECTURA DE MEMORIA Y EL BOOTLOADER

Para comprender a fondo cómo optimizar el firmware, es fundamental conocer la arquitectura de memoria interna de un microcontrolador. A diferencia de un sistema de propósito general, un microcontrolador integra múltiples tecnologías de almacenamiento físico en un mismo chip de silicio, organizadas de acuerdo con su función y volatilidad.

2.1 Clasificación de Memorias Físicas y sus Funciones

Típicamente, un microcontrolador posee tres categorías principales de almacenamiento de datos y programa:

MEMORIA FLASH (MEMORIA DE PROGRAMA):

Es un tipo de memoria no volátil (los datos se conservan al apagar el chip). Su función principal es almacenar el código binario de la aplicación (firmware) generado tras la compilación y datos constantes que no cambian en tiempo de ejecución (como tablas de búsqueda o LUTs, constantes de calibración y cadenas de texto plano). Su proceso de lectura por parte de la CPU es veloz, pero el proceso de escritura o borrado es significativamente más lento y tiene una vida útil limitada en ciclos de escritura (típicamente entre 10,000 y 100,000 ciclos).

MEMORIA RAM (SRAM - MEMORIA DE DATOS):

Es una memoria volátil (pierde absolutamente toda la información en cuanto se corta la alimentación). Su función es almacenar las variables temporales del programa en ejecución, la pila de llamadas (Stack), las variables globales y estáticas que pueden modificarse en caliente, el montón (Heap) y buffers de comunicación. Es extremadamente rápida tanto para lectura como para escritura aleatoria, y no sufre degradación por ciclos de uso. Su tamaño físico es mucho menor que el de la memoria Flash (suele representar entre el 5% y el 10% de esta).

MEMORIA EEPROM:

Es un bloque no volátil dedicado a almacenar parámetros de configuración del usuario, registros de eventos (logs) o valores de calibración que deben sobrevivir al apagado de la máquina, pero que a su vez se modifican periódicamente por el software durante su operación ordinaria. Ofrece la ventaja de poder escribirse y borrarse byte por byte en tiempo de ejecución con una alta resistencia al desgaste (hasta 1,000,000 de ciclos).

Emulación de EEPROM en Flash

Muchos microcontroladores modernos (como la mayoría de la familia STM32) carecen de una memoria EEPROM física dedicada por razones de reducción de coste en el silicio. En su lugar, se reserva un sector de la memoria Flash del usuario y se implementa por software un algoritmo de emulación de EEPROM (usualmente escribiendo en páginas alternas y aplicando técnicas de nivelación de desgaste o *wear leveling*) para imitar el comportamiento de una EEPROM real.

2.1.1 Memorias Auxiliares del Sistema

Adicionalmente a los tres bloques principales, el microcontrolador dispone de otras tecnologías de almacenamiento con propósitos muy específicos:

- **Registros del Núcleo (CPU Registers):** Celdas de memoria de velocidad extrema integradas dentro del procesador, utilizadas para contener operandos aritméticos temporales y el estado del núcleo de manera directa (ej. acumulador, contador de programa, puntero de pila).
- **Boot ROM / Memoria de Sistema (System Memory):** Zona no volátil protegida contra escritura grabada por el fabricante del silicio. Contiene el cargador de arranque inicial (bootloader de fábrica) seguro.
- **OTP (One-Time Programmable):** Sección de la Flash que solo puede escribirse una única vez. Se emplea habitualmente para almacenar identificadores únicos del dispositivo (UID), claves de descifrado y firmas de seguridad.
- **Caché (ej. I-Cache y D-Cache):** Memoria de tamaño muy reducido y alta velocidad que actúa de puente entre el núcleo y la Flash/SRAM en procesadores avanzados de alta gama (como ARM Cortex-M7), acelerando la velocidad de acceso y ejecución de instrucciones.

2.2 El Bootloader y su Relación con la Memoria Flash

El **bootloader** (gestor de arranque) es un código ejecutable de tamaño reducido que se sitúa en el primer peldaño de la ejecución del microcontrolador tras salir de un estado de Reset o encendido, actuando como un intermediario antes del firmware principal del usuario.

Su función principal es evaluar si el sistema debe entrar en un modo de programación (actualización de firmware) o si debe saltar de inmediato a la aplicación principal guardada en memoria. Para decidirlo, suele monitorizar una condición de hardware (por ejemplo, el estado físico de un pin GPIO como el pin B00T0 en STM32 al encender el chip) o una instrucción enviada por puerto serie.

⚠ Relación Crítica con la Memoria Flash

El bootloader opera en estrecha sinergia con la memoria Flash de dos maneras esenciales:

1. **Ubicación de almacenamiento:** Para poder iniciarse tras el encendido, el bootloader debe guardarse en memoria no volátil. Un **bootloader de fábrica** vive en la *System Memory* (ROM grabada por el fabricante), mientras que un **bootloader personalizado de usuario** se almacena en la primera sección direccionable de la memoria Flash del usuario (típicamente desde la base 0x08000000 en STM32), ocupando los primeros sectores.
2. **Modificación de la Flash (Escritura):** En caso de que se solicite una actualización, el bootloader escucha los datos del nuevo firmware a través de puertos de comunicación (UART, SPI, USB, CAN, etc.), y ejecuta comandos para **borrar** físicamente los sectores de la Flash donde reside la aplicación anterior y luego **escribir** el nuevo binario. Una vez verificado el Checksum, cambia el vector de interrupciones y transfiere el flujo de ejecución a la nueva sección de la Flash.

3 BUENAS PRÁCTICAS PARA LA GESTIÓN DE MEMORIA RAM

La memoria RAM es el recurso más escaso en sistemas embebidos. Almacena variables de estado, la pila de llamadas del sistema (Stack), variables globales y estáticas.

3.1 Prohibición Absoluta de la Asignación Dinámica

En el desarrollo de software para PC, la asignación dinámica de memoria mediante funciones como `malloc()`, `calloc()`, `realloc()` y `free()` es habitual. Sin embargo, en sistemas embebidos de tiempo real y recursos limitados, **esta práctica está estrictamente prohibida** por normativas de seguridad crítica como MISRA C.

3.1.1 Consecuencias de la asignación dinámica

- **Fragmentación de Memoria:** Con el uso repetido de reservar y liberar memoria, el mapa de la RAM física se fragmenta. Aparecen pequeños huecos de memoria libre inutilizables porque no son contiguos. Eventualmente, una solicitud de reserva fallará aunque la suma total de memoria libre teórica sea suficiente.
- **Fugas de Memoria (Memory Leaks):** Si un bloque no se libera correctamente (por ejemplo, ante una interrupción o una salida prematura de función), el sistema se queda sin RAM gradualmente hasta congelarse.
- **Falta de Determinismo en Tiempo:** El tiempo que le toma al administrador de memoria buscar un bloque libre en el bloque dinámico (*Heap*) no es constante (complejidad $O(N)$ en el peor de los casos al buscar en la lista ligada de bloques libres), lo cual es inaceptable en sistemas de control de tiempo real con restricciones estrictas de temporización.

⚠ Alternativa en Embebidos

Se debe emplear únicamente **asignación estática** (variables globales o estáticas declaradas en tiempo de compilación) o **asignación en el Stack** (variables locales temporales). Todo el consumo de RAM debe ser predecible y determinable en el momento de la compilación.

Para ilustrar esto, considere el siguiente ejemplo de código inaceptable en sistemas embebidos, donde se usa `malloc()` para un buffer de procesamiento de datos:

```

1 // INCORRECTO: Uso peligroso de asignación dinámica
2 void ProcessSensorData(uint8_t *rawData, uint16_t length)
3 {
4     // malloc() puede fallar o fragmentar la memoria RAM
5     uint8_t *tempBuffer = (uint8_t *)malloc(length * sizeof(uint8_t));
6     if (tempBuffer == NULL)
7     {
8         // Manejo de error complejo
9         return;
10    }
11
12    // Procesamiento...
13    memcpy(tempBuffer, rawData, length);
14
15    free(tempBuffer); // Si ocurre una interrupción o retorno anticipado, fuga de memoria
16 }

```

La alternativa recomendada para eliminar este riesgo es utilizar buffers de tamaño estático máximo conocidos en tiempo de compilación o buffers locales en el Stack si el tamaño es pequeño:

```

1 // CORRECTO: Asignación estática pre-asignada
2 #define MAX_BUFFER_SIZE 256
3
4 static uint8_t s_tempBuffer[MAX_BUFFER_SIZE]; // Asignado en la sección .bss en compilación
5
6 void ProcessSensorData(const uint8_t *rawData, uint16_t length)
7 {
8     if (length > MAX_BUFFER_SIZE)
9     {
10         length = MAX_BUFFER_SIZE; // Truncado seguro
11     }
12
13    // Procesamiento seguro sin riesgo de fragmentación o fallo de reserva
14    memcpy(s_tempBuffer, rawData, length);
15 }

```

3.2 Optimización de Tipos de Datos y Ancho de Palabra (<stdint.h>)

El uso de tipos estándar tradicionales de C como `int`, `short` o `long` debe ser descartado, ya que su longitud varía según la arquitectura del procesador (un `int` suele ocupar 4 bytes en arquitecturas ARM de 32 bits, pero 2 bytes en microcontroladores AVR de 8 bits).

Se debe importar siempre la cabecera estándar `<stdint.h>` y hacer uso exclusivo de tipos con longitud explícita (ver Tabla 1).

Tipo de Dato	Descripción	Rango de Valores	Tamaño (Bytes)
<code>uint8_t</code>	Entero sin signo	0 a 255	1
<code>int8_t</code>	Entero con signo	-128 a 127	1
<code>uint16_t</code>	Entero sin signo	0 a 65,535	2
<code>int16_t</code>	Entero con signo	-32,768 a 32,767	2
<code>uint32_t</code>	Entero sin signo	0 a 4,294,967,295	4
<code>int32_t</code>	Entero con signo	-2,147,483,648 a 2,147,483,647	4

Tabla 1: Tipos de datos de ancho fijo estándar.

i Optimización de Tipos Rápidos (Fast Types)

En procesadores de 32 bits (como ARM Cortex-M), el acceso a registros es óptimo a 32 bits. Si declaras una variable local para un bucle como `uint8_t`, el compilador puede tener que generar instrucciones adicionales de conversión o máscara (como `UXTB` o `AND`) para limpiar los bits superiores del registro tras cada operación aritmética.

Para variables locales de contadores y bucles donde el tamaño no es restrictivo en memoria, se recomienda el uso de tipos rápidos como `uint_fast8_t` o `uint_fast16_t`, los cuales se compilarán al tamaño nativo más eficiente del procesador (32 bits en ARM), optimizando la velocidad de ejecución.

3.3 Estructuras y Alineación de Memoria

Los microcontroladores modernos de 32 bits acceden a la memoria RAM de manera más eficiente en direcciones alineadas (múltiplos de 4 bytes). Si se definen estructuras de datos sin prestar atención al orden de sus variables, el compilador insertará automáticamente bytes de relleno invisibles (*padding*) para forzar la alineación, desperdiciando valiosa memoria RAM.

Considere el siguiente ejemplo de una estructura sin optimizar:

```
1 // Estructura mal alineada (Total: 12 bytes en memoria debido al padding)
2 typedef struct
3 {
4     uint8_t status;      // 1 byte + 3 bytes de padding invisible
5     uint32_t timestamp; // 4 bytes (debe alinearse a direccion multiplo de 4)
6     uint16_t value;     // 2 bytes + 2 bytes de padding al final de la estructura
7 } SensorData_t;
```

La distribución física en la memoria RAM de 32 bits para esta estructura se visualiza en la Tabla 2:

Dirección Relativa	Byte 3	Byte 2	Byte 1	Byte 0
0x00	Padding	Padding	Padding	status (1B)
0x04	timestamp (4 bytes)			
0x08	Padding	Padding	value (2B)	

Tabla 2: Mapa de memoria de estructura con relleno (12 bytes ocupados).

Para solucionar esta ineficiencia de almacenamiento, se deben seguir las siguientes dos directivas:

1. **Ordenar los campos por tamaño descendente:** Colocar primero los elementos más grandes y con restricciones de alineación más severas:
 - **64 bits (8 bytes):** Variables de tipo `double` y enteros de 64 bits (`uint64_t`, `int64_t`). De acuerdo con el estándar de llamada de ARM (AAPCS), requieren alineación a direcciones múltiplos de 8 bytes.
 - **32 bits (4 bytes):** Variables de tipo `float` y enteros de 32 bits (`uint32_t`, `int32_t`). Requieren alineación a direcciones múltiplos de 4 bytes.
 - **16 bits (2 bytes):** Variables de tipo `uint16_t` y `int16_t`. Requieren alineación a direcciones múltiplos de 2 bytes.
 - **8 bits (1 byte):** Variables de tipo `uint8_t`, `int8_t`, `char` y el tipo booleano nativo `_Bool` (o `bool` de `<stdbool.h>`). Tienen una alineación de 1 byte (sin restricciones). Se colocan siempre al final para rellenar los huecos restantes antes del cierre de la estructura.

Esto minimiza drásticamente la necesidad de relleno automático (*padding*) por parte del compilador.

2. **Empaquetar estructuras:** Si el ahorro de RAM es absolutamente crítico (a costa de una pequeña penalización en ciclos de reloj debido a accesos no alineados del procesador), se puede instruir al compilador a empaquetar la estructura eliminando el relleno mediante el atributo `__attribute__((packed))`.

Considere la estructura ordenada y optimizada:

```

1 // Estructura optimizada por orden descendente (Total: 8 bytes en memoria)
2 typedef struct
3 {
4     uint32_t timestamp; // 4 bytes
5     uint16_t value;      // 2 bytes
6     uint8_t status;      // 1 byte + 1 byte de padding al final de la estructura
7 } SensorDataOpt_t;

```

La estructura optimizada ocupa únicamente 8 bytes en memoria en lugar de 12 bytes, reduciendo el consumo en un 33%.

3.4 Prevención de Desbordamiento del Stack y Análisis de Uso

La pila (Stack) es una zona de la memoria RAM reservada para la ejecución de funciones: almacena las direcciones de retorno, los valores de los registros del procesador a restaurar y todas las variables locales no estáticas.

✖ Peligro Crítico: Stack Overflow

Si el Stack crece excesivamente hacia direcciones de memoria inferiores e invade el espacio asignado a variables globales y estáticas (sección `.data` y `.bss`), ocurre un desbordamiento del Stack (*Stack Overflow*). Al carecer la mayoría de los microcontroladores de un sistema operativo o unidad de protección activa (MPU) configurada, esto corrompe los datos globales de forma silenciosa, resultando en cuelgues, reinicios abruptos o fallos de tipo *HardFault*.

Para prevenir esta situación extrema, se deben adoptar las siguientes directivas de diseño:

- **No declarar arreglos grandes localmente:** Si se requiere un buffer temporal de comunicación (por ejemplo, 512 bytes), nunca debe declararse dentro de una función de forma local. Use la palabra clave `static` para asignarla a la sección estática de RAM global, o use un buffer global reutilizable.
- **Prohibición estricta de recursividad:** Las llamadas a funciones recursivas empujan nuevos marcos de ejecución en el Stack por cada iteración. En firmware embebido, todo algoritmo debe implementarse de manera estrictamente iterativa.
- **Monitoreo en el Linker Script:** El archivo de enlace (por ejemplo, `STM32F303XX_FLASH.ld`) define explícitamente el tamaño mínimo reservado para el Stack. Verifique que la variable `_Min_Stack_Size` sea suficiente para las necesidades del sistema.

ℹ Análisis Estático del Stack con GCC

El compilador GCC ofrece herramientas poderosas para calcular el uso máximo del Stack en tiempo de compilación. Añadiendo la bandera de compilación `-fstack-usage`, el compilador generará un archivo complementario con extensión `.su` por cada archivo de código fuente compilado. Este archivo detalla el tamaño exacto en bytes de la pila utilizado por cada una de las funciones definidas en el proyecto.

3.4.1 Habilitación de `-fstack-usage` en CMake y VS Code

Para activar la generación de los reportes del Stack en un entorno de desarrollo basado en VS Code y CMake, se debe agregar el flag al archivo `CMakeLists.txt` principal. Existen dos alternativas estándar en CMake:

1. **Mediante `target_compile_options` (Recomendado):** Asocia la bandera específicamente a tu objetivo ejecutable. En el archivo `CMakeLists.txt` raíz, localiza la definición del proyecto y añade el comando después de que el objetivo haya sido creado mediante `add_executable`:

```
1 # Agregar bandera de uso de pila al target del proyecto
2 target_compile_options(${CMAKE_PROJECT_NAME} PRIVATE -fstack-usage)
```

2. **Mediante `add_compile_options` (Global):** Si deseas aplicar la bandera de forma global a todos los objetivos y subdirectorios de CMake, puedes declararla en las primeras líneas de tu script:

```
1 # Opcion global para todos los archivos compilados en el directorio
2 add_compile_options(-fstack-usage)
```

Una vez configurado y recompilado el proyecto (usando el botón **Build** en VS Code), los archivos con extensión `.su` se generarán dentro del directorio de construcción temporal de CMake.

Típicamente se ubican en la ruta:

`build/Debug/CMakeFiles/<NombreProyecto>.dir/Core/Src/main.c.su` (o la carpeta correspondiente a cada archivo fuente).

Cada línea de un archivo `.su` contiene un formato sencillo como el siguiente:

```
1 main.c:104:main    24    static
2 main.c:236:Error_Handler    0    static
```

Esto indica que la función `main()` en el archivo `main.c` (declarada en la línea 104) requiere exactamente 24 bytes de memoria en la pila de forma estática.

3.5 Paso de Parámetros por Referencia (Punteros) y Uso de Const

Cuando una función requiere el acceso a estructuras complejas, arreglos de gran tamaño o instancias de controladores de periféricos (como `OPT3001_HandleTypeDef`), pasar los datos "por valor" provoca que el compilador copie la estructura completa en el Stack, aumentando drásticamente el consumo de pila y los ciclos de CPU.

Se debe pasar el parámetro **por referencia** mediante un puntero. Si la función únicamente leerá los datos del objeto, se debe calificar la variable con el modificador `const` para garantizar la seguridad de solo lectura del objeto:

```
1 // INCORRECTO: Copia toda la estructura (ej. 24 bytes) en el Stack
2 void OPT3001_Update(OPT3001_HandleTypeDef opt3001);
3
4 // CORRECTO: Pasa un puntero (solo copia 4 bytes de direccion)
5 void OPT3001_Update(OPT3001_HandleTypeDef *opt3001);
6
7 // RECOMENDADO (Paso por referencia de solo lectura seguro):
8 void OPT3001_Read(const OPT3001_HandleTypeDef *opt3001);
```

4 OPTIMIZACIÓN DE ALMACENAMIENTO FLASH (ROM) Y RENDIMIENTO

La memoria Flash contiene el código ejecutable del firmware generado y los datos constantes de solo lectura que deben persistir tras cortes de energía.

4.1 Almacenamiento de Constantes en Flash mediante `const`

En C, cualquier cadena de texto constante o tabla de consulta (*look-up table*) se copiará por defecto a la memoria RAM de lectura/escritura durante el arranque a menos que se use explícitamente el calificador `const`. Esto duplica el espacio consumido (ocupa Flash para el valor inicial y RAM en ejecución).

En arquitecturas ARM Cortex-M, calificar estas estructuras como `const` le indica al compilador que mantenga las variables exclusivamente en la sección de solo lectura de la memoria Flash (`.rodata`), liberando espacio crítico de RAM.

```

1 // INCORRECTO: Ocupa 16 bytes de Flash y 16 bytes de RAM en tiempo de ejecucion
2 uint16_t SinusoidalLUT[8] = {0, 707, 1000, 707, 0, -707, -1000, -707};
3
4 // CORRECTO: Se almacena exclusivamente en la seccion .rodata (Flash/ROM). RAM libre = 0
   bytes:
5 const uint16_t SinusoidalLUT[8] = {0, 707, 1000, 707, 0, -707, -1000, -707};

```

El Proceso de Inicialización y el Vector de Arranque

Durante el arranque del microcontrolador, antes de ingresar a la función `main()`, se ejecuta el código de inicialización en ensamblador (por ejemplo, en `startup_stm32f303xe.s`). Este código realiza la copia física de todos los valores de inicialización de variables de la memoria Flash a la memoria RAM (sección `.data`), e inicializa a cero la sección `.bss`. Al definir variables como `const`, el compilador las ubica directamente en la sección `.rodata`. Dado que el procesador puede leer directamente desde la memoria Flash, no es necesario copiarlas a RAM, eliminando el consumo de esta y reduciendo la duración del proceso de arranque.

4.2 Evitar la Aritmética de Punto Flotante y Funciones Pesadas

Muchos microcontroladores embebidos de bajo costo (como los basados en ARM Cortex-M0 o Cortex-M3) carecen de una Unidad de Punto Flotante (FPU) física en su silicio. Cuando realizamos operaciones matemáticas con variables de tipo `float` o `double`, el compilador se ve obligado a enlazar e inyectar extensas rutinas matemáticas de emulación por software en la memoria Flash, lo cual infla severamente el tamaño del archivo binario ejecutable y ralentiza el desempeño de la CPU por órdenes de magnitud.

4.2.1 Reglas para optimizar cálculos aritméticos:

1. **Uso de Aritmética de Punto Fijo:** En lugar de utilizar números decimales nativos, multiplique los valores reales por factores de escala de potencias de 10 (10, 100, 1000) o potencias de 2 y realice las operaciones aritméticas utilizando enteros estándar. Por ejemplo, si se desea calcular el voltaje de entrada a partir de una lectura de un ADC de 12 bits con referencia de 3.3V, se puede evitar el punto flotante de la siguiente manera:

```

1 // Enfoque ineficiente con Punto Flotante
2 float voltage = (float)adcRead * 3.3f / 4095.0f; // Requiere emulacion float
3
4 // Enfoque optimizado con Punto Fijo (Voltaje en milivoltios)
5 uint32_t voltage_mv = ((uint32_t)adcRead * 3300) / 4095; // Aritmetica entera rapida

```

2. **Evitar funciones de la biblioteca matemática estándar (math.h):** Funciones como `pow()`, `sin()`, `cos()` y `log()` arrastran dependencias sumamente complejas que incrementan drásticamente el peso del firmware.

- *Desplazamientos de bits:* Para resolver potencias de 2 (como $2^{\text{exponente}}$ requeridas para decodificar los bits de mantisa y exponente en sensores como el sensor de luz ambiental OPT3001), nunca use `pow(2, exponente)`. Reemplázelo por un desplazamiento lógico a la izquierda:

```

1 // INCORRECTO: Lento y requiere enlazar libreria matematica pesada
2 double lux = 0.01 * pow(2, exponente) * mantisa;
3
4 // CORRECTO: Resuelto en un solo ciclo de reloj por la CPU mediante hardware de
5 //           barrido
6 double lux = 0.01 * (double)(1 << exponente) * (double)mantisa;
```

- *Tablas de Consulta (LUT):* Si requiere calcular funciones trigonométricas complejas en sistemas en tiempo real, genere previamente una tabla constante (LUT) indexada y guardada en Flash mediante el calificador `const`. Para mayor precisión sin aumentar el tamaño de la tabla, puede implementar una interpolación lineal entre los dos puntos más cercanos:

```

1 // Seno de 0 a 90 grados en pasos de 1 grado, escalado por 1000 (punto fijo)
2 // Guardado en .rodata (Flash/ROM). Solo consume 182 bytes de Flash y 0 bytes de
3 //   RAM:
4 const int16_t SinLUT_x1000[91] = {
5     0,  17,  35,  52,  70,  87, 105, 122, 139, 156,
6     174, 191, 208, 225, 242, 259, 276, 292, 309, 326,
7     342, 358, 375, 391, 407, 423, 438, 454, 469, 485,
8     500, 515, 530, 545, 559, 574, 588, 602, 616, 629,
9     643, 656, 669, 682, 695, 707, 719, 731, 743, 755,
10    766, 777, 788, 799, 809, 819, 829, 839, 848, 857,
11    866, 875, 883, 891, 899, 906, 914, 921, 927, 934,
12    940, 946, 951, 956, 961, 966, 970, 974, 978, 982,
13    985, 988, 990, 993, 995, 996, 998, 999, 999, 1000,
14    1000
15 };
16
17 // Obtiene el seno utilizando interpolacion lineal
18 int16_t GetSine_FixedPoint(float angleDegrees)
19 {
20     if (angleDegrees < 0.0f) angleDegrees = 0.0f;
21     if (angleDegrees > 90.0f) angleDegrees = 90.0f;
22
23     uint8_t index = (uint8_t)angleDegrees;
24     if (index >= 90) return SinLUT_x1000[90];
25
26     float fractional = angleDegrees - (float)index;
27
28     // Leer los dos puntos adyacentes de la LUT
29     int16_t y0 = SinLUT_x1000[index];
30     int16_t y1 = SinLUT_x1000[index + 1];
31
32     // Interpolacion lineal: y = y0 + (y1 - y0) * fraccion
33     return y0 + (int16_t)((float)(y1 - y0) * fractional);
34 }
```

Explicación del funcionamiento:

- **Eficiencia en memoria:** La tabla `SinLUT_x1000` tiene 91 elementos de tipo `int16_t` (2 bytes cada uno). Al ser calificada con `const`, se almacena directamente en la Flash ocupando únicamente 182 bytes, lo que deja la valiosa memoria RAM intacta.
- **Interpolación Lineal:** Si llamamos a `GetSine_FixedPoint(45.5f)`, el índice entero será 45 y la fracción será 0.5f. El código leerá `SinLUT_x1000[45]` (707) y `SinLUT_x1000[46]` (719). La interpolación calculará:

$$\text{Resultado} = 707 + (719 - 707) \times 0.5 = 707 + 6 = 713$$

Esto representa un valor de 0.713 (el valor real con alta precisión es $\sin(45.5^\circ) \approx 0.71325$), logrando una excelente aproximación sin llamar a la pesada biblioteca trigonométrica de GCC.

5 USO DEL CALIFICADOR `volatile`

En el desarrollo de software de escritorio, el compilador realiza optimizaciones agresivas asumiendo que los valores de las variables en memoria RAM únicamente cambian si el propio código principal del hilo de ejecución ejecuta una instrucción directa de escritura sobre ellas. En sistemas embebidos, esta suposición básica suele provocar fallos críticos de funcionamiento.

El calificador **`volatile`** le indica explícitamente al optimizador del compilador que la variable asociada puede cambiar de valor de manera externa en cualquier instante, debido a eventos del hardware (como registros de periféricos) o asíncronos (como rutinas de interrupción). Por lo tanto, el compilador tiene estrictamente prohibido optimizar el acceso a dicha variable almacenando su valor en un registro interno de la CPU; en su lugar, está obligado a forzar una lectura física desde la dirección de memoria RAM en cada instrucción del programa.

5.1 ¿Cuándo debe utilizarse?

1. **Variables globales compartidas modificadas en Rutinas de Interrupción (ISR):** Si una bandera se define de manera convencional, el compilador optimizará el bucle en el programa principal guardando la lectura del valor en un registro de la CPU de manera permanente. Dado que la interrupción modifica la variable en la memoria física y no en el registro de la CPU asignado al bucle, el programa principal nunca se enterará del cambio, quedando atrapado en un bucle infinito.

Considere la diferencia generada por el compilador en el código de máquina (ver código en Lista 1):

Listing 1: Ejemplo de uso de variable `volatile`.

```

1 // Variable global modificada por hardware/interrupciones
2 volatile uint8_t g_systemReady = 0;
3
4 void EXTI0_IRQHandler(void)
5 {
6     g_systemReady = 1; // Se ejecuta de forma asincrónica ante interrupción de pin
7 }
8
9 int main(void)
10 {
11     // ... inicialización ...
12     while(!g_systemReady)
13     {
14         // Si no fuera volatile, el compilador optimizaría el bucle,
15         // cargando g_systemReady en un registro una sola vez.
16     }
17     // Continuar ejecución...
18 }
19
```

2. **Registros de periféricos de Entrada/Salida mapeados en memoria:** Los periféricos cambian de estado físicamente en el chip (por ejemplo, el bit RXNE del registro de estado UART se establece en 1 cuando llega un carácter). Las definiciones de registros provistas por el fabricante (dentro de las librerías CMSIS) utilizan el calificador `volatile` de manera interna en todas sus macros para garantizar la lectura de estado del hardware en tiempo real.

5.2 Análisis del código Ensamblador generado

Para comprender de forma profunda el efecto del calificador, considere la traducción al ensamblador ARM Cortex-M generado por el compilador para el bucle de espera anterior:

Listing 2: Ensamblador generado sin calificar la variable como volatile.

```

1 ; --- SIN CALIFICAR COMO VOLATILE ---
2     LDR R1, =g_systemReady ; R1 almacena la direccion de memoria de la variable
3     LDR R0, [R1]           ; Carga el valor inicial de g_systemReady en el registro R0
4 .L2:
5     CMP R0, #0             ; Compara el valor en R0 con cero
6     BEQ .L2               ; Si es cero, salta a .L2 (Bucle infinito en registros!)
7                             ; La memoria fisica NUNCA se vuelve a leer.

```

Listing 3: Ensamblador generado al calificar la variable como volatile.

```

1 ; --- CALIFICADO COMO VOLATILE (Comportamiento correcto) ---
2     LDR R1, =g_systemReady ; R1 almacena la direccion de memoria de la variable
3 .L2:
4     LDR R0, [R1]           ; Carga el valor actual de g_systemReady desde la RAM en R0
5     CMP R0, #0             ; Compara el valor en R0 con cero
6     BEQ .L2               ; Si es cero, salta a .L2
7                             ; Fuerza la lectura desde RAM en cada iteracion del bucle.

```

6 OPTIMIZACIÓN DEL COMPILADOR GCC EN VS CODE

El compilador de C/C++ utilizado de manera estándar para microcontroladores ARM (la suite de herramientas `arm-none-eabi-gcc`) dispone de diversos niveles de optimización de código, los cuales se seleccionan en la etapa de compilación mediante parámetros o banderas de la terminal.

6.1 Niveles de Optimización en GCC

Los distintos niveles de optimización controlan el balance entre la velocidad de ejecución, el espacio físico del binario resultante en la memoria Flash y la facilidad para realizar la depuración interactiva en tiempo real. En la Tabla 3 se presenta una comparativa de estos niveles:

Flag	Opt. Tamaño	Opt. Velocidad	Depuración	Descripción
-O0	Ninguna	Ninguna	Excelente	Traduce directamente C a ensamblador paso a paso.
-O1	Moderada	Moderada	Aceptable	Optimización básica que no altera el orden de instrucciones.
-O2	Alta	Alta	Compleja	Aplica optimizaciones estándar sin incrementar tamaño.
-O3	Baja	Muy Alta	Muy Difícil	Optimización agresiva (desenrolla bucles, inlining).
-Os	Máxima	Alta	Compleja	Recomendada. Reduce al mínimo el binario en Flash.
-Ofast	Baja	Extrema	Imposible	Activa optimizaciones rápidas de flotantes sin estándar.

Tabla 3: Comparativa de niveles de optimización de GCC.

6.2 La Bandera de Optimización de Tamaño: -Os

La bandera -Os (Optimize for Size) es la opción estándar de la industria y la mejor práctica recomendada para el firmware de producción en microcontroladores con recursos limitados.

¿Cómo opera la bandera -Os bajo el capó?

La optimización -Os activa todas las optimizaciones estándar de -O2 que no impliquen un aumento del tamaño del código de máquina resultante. Adicionalmente, realiza análisis específicos orientados a disminuir el espacio en bytes ocupado por el binario:

- **Fusión de bloques idénticos:** Detecta segmentos de código ensamblador repetitivos a lo largo del firmware y los unifica en subrutinas compartidas.
- **Oclusión de Frame Pointer:** Elimina el puntero de marco de pila en funciones donde no sea estrictamente necesario (`-fomit-frame-pointer`), liberando un registro de la CPU para otras variables y reduciendo el tamaño del marco de pila.
- **Desactivación de desenrollado de bucles (*Loop Unrolling*):** A diferencia de -O3, que duplica las instrucciones de un ciclo en línea para acelerarlo, -Os mantiene los bucles compactos para no duplicar líneas de código.

6.3 Cómo configurar la optimización en entornos de VS Code

Dependiendo de la estructura del proyecto de firmware y del sistema de construcción que se utilice dentro del editor Visual Studio Code, la optimización `-Os` se debe configurar de acuerdo con los siguientes escenarios prácticos:

6.3.1 Caso A: Proyectos basados en CMake (STM32 VS Code Extension)

En las nuevas herramientas oficiales de STMicroelectronics basadas en CMake, la optimización y generación del binario se configuran mediante perfiles de CMake (*Presets*). Para forzar o configurar la compilación de tamaño, existen dos alternativas:

1. **Mediante CMakePresets.json (Recomendado):** El archivo `CMakePresets.json` en la raíz del proyecto define los perfiles de compilación. Para activar la compilación optimizada en tamaño, asegúrese de definir o seleccionar el preset de configuración configurado al tipo de compilación `MinSizeRel` (Minimal Size Release), el cual inyecta internamente la bandera `-Os`.

Un ejemplo del fragmento de configuración en `CMakePresets.json` se muestra en el Listado 4:

Listing 4: Fragmento de configuración en `CMakePresets.json`.

```

1 {
2   "version": 3,
3   "configurePresets": [
4     {
5       "name": "MinSizeRel",
6       "displayName": "Optimizado para Tamano",
7       "description": "Compilacion en modo Release con optimizacion -Os",
8       "binaryDir": "${sourceDir}/build/MinSizeRel",
9       "generator": "Ninja",
10      "cacheVariables": {
11        "CMAKE_BUILD_TYPE": "MinSizeRel",
12        "CMAKE_TOOLCHAIN_FILE": "cmake/gcc-arm-none-eabi.cmake"
13      }
14    }
15  ]
16 }
```

2. **Forzar la bandera directamente en CMakeLists.txt:** Si desea que el compilador aplique la optimización `-Os` sin importar el tipo de preset seleccionado por el usuario, agregue la directiva `add_compile_options` o defínala en las opciones de compilación del proyecto principal dentro de su archivo `CMakeLists.txt` raíz:

```

1 # Forzar la optimizacion de tamano y habilitar simbolos de depuracion
2 add_compile_options(-Os -g)
```

O bien, asocie la bandera específicamente a la configuración de lanzamiento o minimización de tamaño:

```

1 # Establecer las banderas especificas para el modo de minimizacion de tamano
2 set(CMAKE_C_FLAGS_MINSIZEREL "-Os -DNDEBUG")
```

6.3.2 Caso B: Proyectos basados en Makefile tradicionales

Si el proyecto se compila llamando de forma directa a la utilidad `make` mediante un archivo `Makefile` generado previamente (por ejemplo, al exportar un proyecto antiguo de CubeMX), la optimización del compilador se declara directamente sobre la variable de configuración de optimización (comúnmente llamada `OPT` o `CFLAGS`):

Listing 5: Configuración de optimización de tamaño en un Makefile.

```

1 #-----
2 # optimization flags
3 #-----
4 # Cambiar -O3 o -O0 por -Os para reducir el tamaño del binario
5 OPT = -Os
6
7 # Banderas del compilador que heredan la variable OPT
8 CFLAGS = $(MCU) $(C_DEFS) $(C_INCLUDES) $(OPT) -Wall -fdata-sections -ffunction-sections

```

6.3.3 Caso C: Tareas integradas de compilación en VS Code (tasks.json)

Si está utilizando Visual Studio Code para compilar un archivo de código fuente de forma directa mediante una tarea manual de compilación (declarada en `.vscode/tasks.json`), debe inyectar de forma directa el parámetro de línea de comandos `-Os` en la lista de argumentos de la cadena de herramientas del compilador GCC (`arm-none-eabi-gcc`):

Listing 6: Configuración del compilador `arm-none-eabi-gcc` en `tasks.json`.

```

1 {
2   "version": "2.0.0",
3   "tasks": [
4     {
5       "type": "shell",
6       "label": "Compilar Proyecto Embebido",
7       "command": "arm-none-eabi-gcc",
8       "args": [
9         "-g",
10        "-Os",           // Bandera de optimización orientada a tamaño
11        "-mcpu=cortex-m4", // Arquitectura del procesador de destino
12        "-mthumb",        // Modo de juego de instrucciones Thumb-2
13        "${file}",        // Archivo actual en edición en el editor
14        "-o",
15        "${fileDirname}/${fileBasenameNoExtension}.elf"
16      ],
17      "group": {
18        "kind": "build",
19        "isDefault": true
20      }
21    }
22  ]
23 }

```

6.4 Banderas adicionales de optimización para reducir tamaño

Para maximizar los efectos del flag `-Os`, se recomienda acompañarlo de banderas avanzadas del compilador y enlazador que eliminan el código muerto (código fuente que nunca es llamado o ejecutado en la aplicación):

- **-ffunction-sections y -fdata-sections:** Indican al compilador GCC que coloque cada función del código y cada objeto de datos estático en su propia sección dedicada en el archivo objeto (`.o`). De forma nativa, todo el código se ubica en una sola sección continua.
- **-Wl,-gc-sections:** Instruye al enlazador (*linker*) a ejecutar una recolección de basura de las secciones de memoria (*garbage collection of sections*). En la fase de enlace, busca todas aquellas secciones independientes creadas en el paso anterior que nunca sean referenciadas en el árbol de llamadas del programa y las elimina por completo del binario final.

El uso combinado de `-Os` junto con estas dos banderas es capaz de reducir el tamaño final del binario en Flash en porcentajes que oscilan entre un 20% y un 50%, permitiendo almacenar firmware complejo en microcontroladores de baja gama.

¿Cómo operan y por qué son tan eficientes juntas?

El funcionamiento y la sinergia de estas banderas se detalla a continuación:

-ffunction-sections y -fdata-sections (COMPILADOR):

Por qué es necesario: Por defecto, el compilador GCC compila todo tu archivo fuente de C (funciones y variables globales) en una sola sección continua de código (llamada `.text` y `.data` respectivamente). Si el linker detecta que no estás usando una sola función de ese archivo, no la puede eliminar porque está empaquetada junto al resto.

Qué hace: Obliga al compilador a dividir el archivo objeto y a colocar cada función en una sección separada (ej. `.text.GetSine_FixedPoint`) y cada variable en otra (ej. `.data.s_tempBuffer`).

-wl,-gc-sections (ENLAZADOR/LINKER):

Qué hace: “gc” significa *Garbage Collection* (recolección de basura). Durante la etapa de enlace (cuando se juntan todos los archivos objeto `.o` para armar el ejecutable `.elf`), el linker escanea el árbol de llamadas partiendo desde el punto de entrada (`Reset_Handler` y `main`).

El resultado: Cualquier sección de función o variable que fue separada en el paso anterior y que no sea alcanzada ni llamada en el flujo de ejecución, es descartada del binario final.

EL IMPACTO REAL:

Esta técnica es crítica al integrar librerías externas completas (como la HAL de STM32Cube o controladores de sensores como el OPT3001). Si la librería ofrece 20 funciones y tú solo utilizas 2, las otras 18 funciones son eliminadas físicamente de la memoria Flash del microcontrolador, logrando reducciones del binario final de entre un 20% y un 50%.

6.4.1 Configuración de eliminación de código muerto en CMake

Para inyectar estas banderas en un proyecto de CMake en VS Code, se deben agregar tanto las opciones de compilación como las de enlace sobre tu objetivo en el archivo `CMakeLists.txt` principal:

```

1 # 1. Separar funciones y datos en secciones independientes (Compilador)
2 target_compile_options(${CMAKE_PROJECT_NAME} PRIVATE
3     -ffunction-sections
4     -fdata-sections
5 )
6
7 # 2. Descartar secciones no referenciadas / código muerto (Enlazador)
8 target_link_options(${CMAKE_PROJECT_NAME} PRIVATE
9     -Wl,--gc-sections
10 )

```

6.5 Análisis del uso de memoria en VS Code

Tras finalizar el proceso de compilación con las banderas de optimización anteriores, es fundamental cuantificar el resultado exacto del uso de memoria física RAM y Flash en el microcontrolador. En Visual Studio Code, esto se visualiza en dos secciones clave:

1. **Salida de Consola del Build:** Al finalizar la compilación basada en CMake, la consola integrada imprimirá una tabla con la región de memoria RAM, FLASH y CCMRAM del microcontrolador especificando los bytes consumidos y el porcentaje del total libre del chip.
2. **Herramientas Gráficas Integradas de ST:** Haciendo clic sobre la extensión de STMicroelectronics en la barra lateral izquierda, puede acceder a las herramientas visuales **STM32 Build Analyzer** y **Memory Inspector**. Estas herramientas examinan la información contenida en el archivo enlazado .elf para mostrar gráficos interactivos con el uso exacto de memoria RAM y Flash detallando el peso en bytes que consume cada función de su programa.